

PROJECT TITLE: Email/SMS Spam Classifier

GROUP MEMBERS:

01	Hazrat Bilal	24PWBCS1345
02	Haroon Ur Rashid	24PWBCS1357
03	Khalida Afghan	24PWBCS1416
04	Ilham Raza	24PWBCS1259

SECTION: E

**DEPARTMENT OF COMPUTER SCIENCE
UNIVERSITY OF ENGINEERING & TECHNOLOGY,
PESHAWAR**

PROJECT REPORT

SUBJECT: INTRODUCTION TO DATA SCIENCE

INSTRUCTOR: RIDA ZARKASH

Table of Contents

Abstract	2
Introduction.....	3
Problem Statement	4
Objectives	5
Methodology	6
Dataset.....	6
1.Data Cleaning	7
2. Exploratory Data Analysis (EDA)	10
1. Class Distribution.....	10
2. Text Length Analysis Three new numerical features were created from each message:	11
3. Descriptive Statistics Statistical summaries (count, mean, standard deviation, minimum, maximum, and quartiles) were generated for:.....	12
4. Character Distribution.....	14
5. Word Distribution.....	15
6. Pair Plot Analysis	16
7. Correlation Heatmap Analysis	17
3. Text Preprocessing	18
4. Feature Extraction	19
a. Spam Message Word Cloud	19
b. Ham Message Word Cloud	20
4. Model Building	22
5. Model Evaluation	23
6. Model Selection.....	24
7. Advanced Model Development	25
8. Model Saving	26
9. Deployment	26
Conclusion	27

Abstract

This project presents an SMS Spam Detection system using Machine Learning and Natural Language Processing (NLP). The objective is to automatically classify SMS messages as Spam or Ham, helping users filter unwanted messages efficiently.

The dataset was cleaned by removing unnecessary columns and duplicate records. Text preprocessing techniques including lowercase conversion, tokenization, stop-word removal, punctuation removal, and stemming were applied to improve text quality.

The processed text was transformed into numerical features using the TF-IDF vectorizer. Three Naive Bayes classifiers were trained and evaluated using accuracy, precision, and confusion matrix.

Among them, the Multinomial Naive Bayes classifier produced the best results and was selected as the final model. The trained model was saved using Pickle and deployed as a Streamlit web application for real-time spam message prediction.

Introduction

Spam messages are unsolicited messages that often contain advertisements, scams, fraudulent offers, or phishing attempts. Due to the increasing number of spam messages, manually filtering them has become difficult and time-consuming.

Machine Learning provides an effective solution by automatically classifying messages based on their content. This project develops an SMS Spam Detection system using Natural Language Processing and Machine Learning techniques.

The text messages are cleaned and converted into numerical features using the TF-IDF vectorization method. Multiple Naive Bayes algorithms are trained and compared to identify the most suitable model.

The Multinomial Naive Bayes classifier achieved the highest performance and was selected for deployment. The final system is implemented as a Streamlit web application that allows users to quickly determine whether an SMS message is Spam or Ham.

Problem Statement

The increasing number of spam SMS messages exposes users to fraud, phishing attacks, and unwanted advertisements. There is a need for an automated system that can accurately classify messages as Spam or Ham to improve communication security.

Objectives

- Build an SMS spam detection model.
- Clean and preprocess text data.
- Perform exploratory data analysis.
- Convert text into TF-IDF features.
- Train and compare Naive Bayes classifiers.
- Evaluate model performance.
- Deploy the model using Streamlit.

Methodology

Dataset

 SMS Spam Collection Dataset

```
df = pd.read_csv("spam.csv", encoding="latin-1")
df.head()
```

✓ 0.0s

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

1.Data Cleaning

The dataset was cleaned to improve its quality and prepare it for machine learning. The following steps were performed:

- Removed unnecessary columns (Unnamed: 2, Unnamed: 3, Unnamed: 4) containing mostly null values.

```
0   v1          5572 non-null   object
1   v2          5572 non-null   object
2   Unnamed: 2    50 non-null   object
3   Unnamed: 3    12 non-null   object
4   Unnamed: 4     6 non-null   object
dtypes: object(5)
memory usage: 217.8+ KB
```

```
# drop last 3 cols as much of it are null
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
```

✓ 0.0s

```
df.sample(5)
```

✓ 0.0s

	v1	v2
1376	ham	Auntie huai juan never pick up her phone
1236	ham	How much are we getting?
4463	ham	Ranjith cal drpd Deeraj and deepak 5min hold
5255	ham	Ok Sweet dreams

- Renamed the columns from v1 and v2 to target and text for better readability.

```
1] ✓ 0.0s  
# renaming the cols  
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)  
df.sample(5)
```

	target	text
3463	ham	My phone
5207	ham	Babe, I'm answering you, can't you see me ? Ma...
4286	ham	I wud never mind if u dont miss me or if u don...
4818	ham	Got smaller capacity one? Quite ex...
3756	spam	YES! The only place in town to meet exciting a...

- Encoded the target labels using Label Encoding (Ham = 0, Spam = 1).

```
1] ✓ 0.0s  
# encode the target column  
from sklearn.preprocessing import LabelEncoder  
encoder = LabelEncoder()
```

```
2] ✓ 0.0s  
# spam=1, ham=0  
df['target'] = encoder.fit_transform(df['target'])
```

```
3] ✓ 0.0s  
df.head()
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

- Checked for missing values (no missing values were found).

```
▶ # missing values
df.isnull().sum()

[14] ✓ 0.0s

... target    0
text        0
dtype: int64
```

- Removed duplicate records to ensure a clean and consistent dataset.

```
# check for duplicate values
df.duplicated().sum()

✓ 0.0s
```

np.int64(403)

```
# remove duplicates
df = df.drop_duplicates(keep='first')

✓ 0.0s
```

```
df.duplicated().sum()

✓ 0.0s
```

np.int64(0)

```
df.shape

✓ 0.0s
```

(5169, 2)

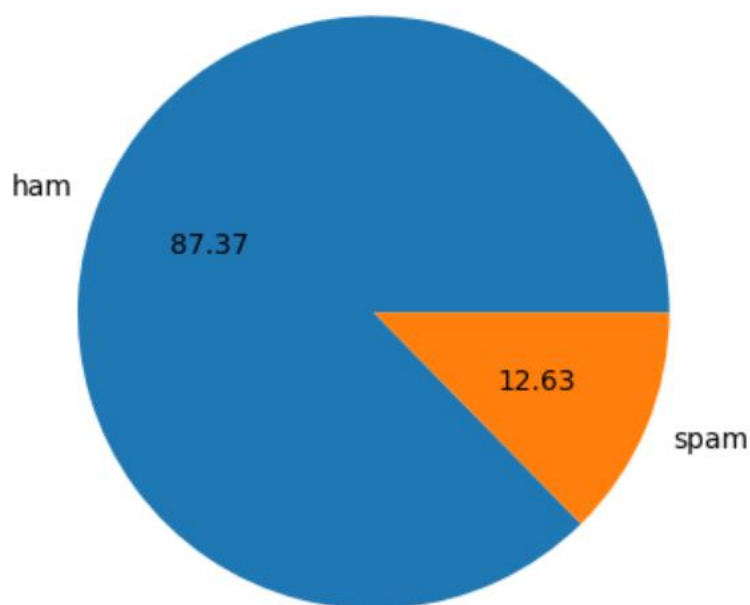
2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the characteristics of the SMS dataset and identify patterns between spam and ham messages before building the machine learning models.

1. Class Distribution

- Counted the number of spam and ham messages.
- Visualized the distribution using a pie chart.

...



- The pie chart shows the distribution of SMS messages in the dataset. Approximately **87.37%** of the messages are **Ham (non-spam)**, while **12.63%** are **Spam**. This indicates that the dataset is **imbalanced**, with ham messages significantly outnumbering spam messages. This imbalance should be considered during model development and evaluation to ensure accurate spam detection.

2. Text Length Analysis

Three new numerical features were created from each message:

- **Number of Characters:** Total characters in each SMS.
- **Number of Words:** Total words in each SMS.
- **Number of Sentences:** Total sentences in each SMS.

```
df.head()
```

✓ 0.0s

	target	text	num_characters	num_words	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2
1	0	Ok lar... Joking wif u oni...	29	8	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	0	U dun say so early hor... U c already then say...	49	13	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1

```
df[['num_characters', 'num_words', 'num_sentences']].describe()
```

3. Descriptive Statistics

Statistical summaries (count, mean, standard deviation, minimum, maximum, and quartiles) were generated for:

● Entire dataset

✓ 0.0s

	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

● Ham messages

8] ✓ 0.0s

	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

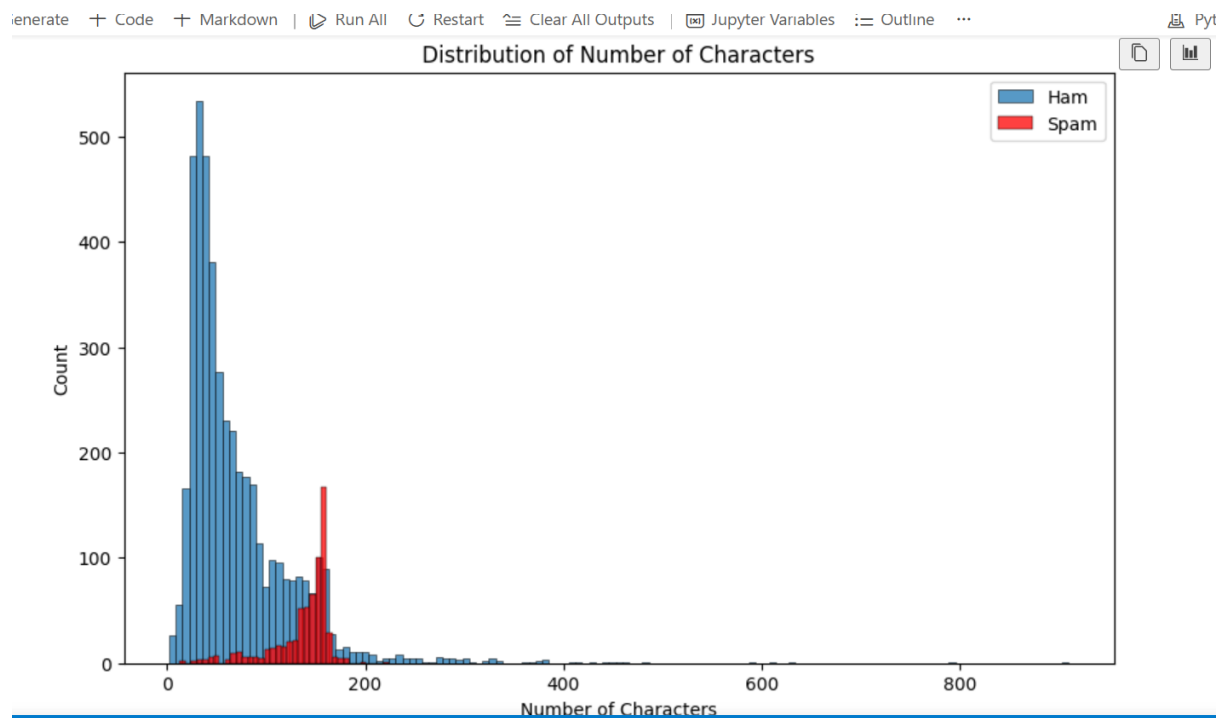
- Spam messages

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425
min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000

This comparison showed that spam messages generally contain more characters, words, and sentences than ham messages.

4. Character Distribution

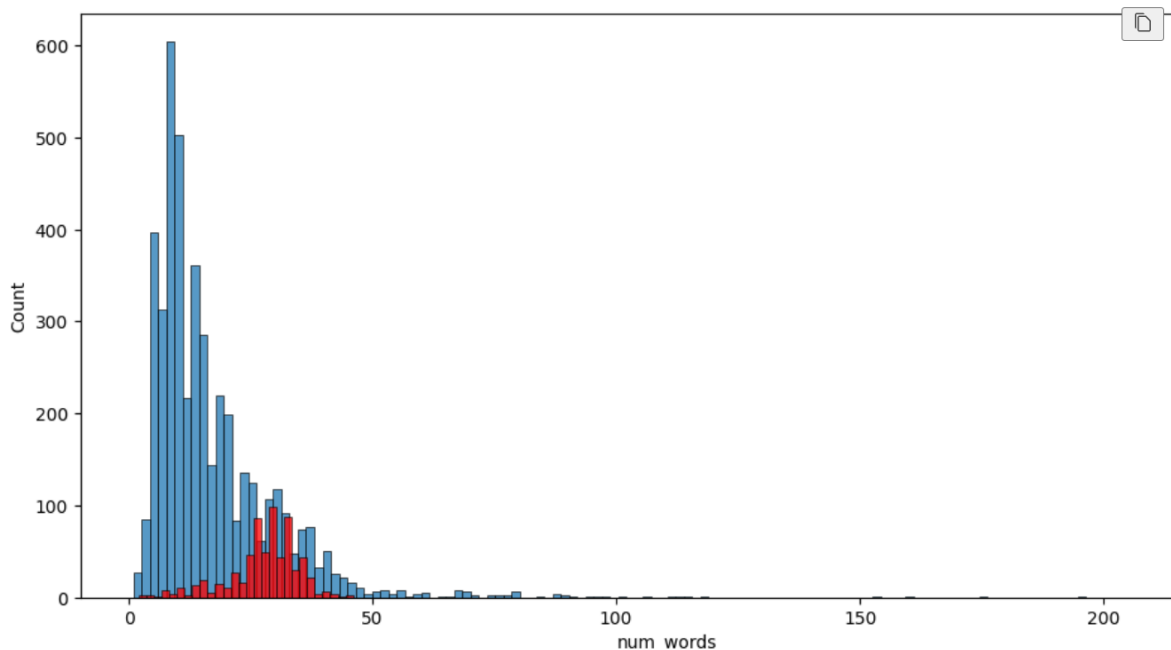
- Histograms were plotted to compare the distribution of the number of characters in spam and ham messages.
- Spam messages were generally longer than ham messages.



The histogram compared the number of characters in Ham and Spam messages. It showed that ham messages were generally shorter, with most containing fewer than 100 characters. In contrast, spam messages were generally longer, with a higher concentration between 100 and 170 characters. This indicated that the number of characters was a useful feature for distinguishing spam from ham messages.

5. Word Distribution

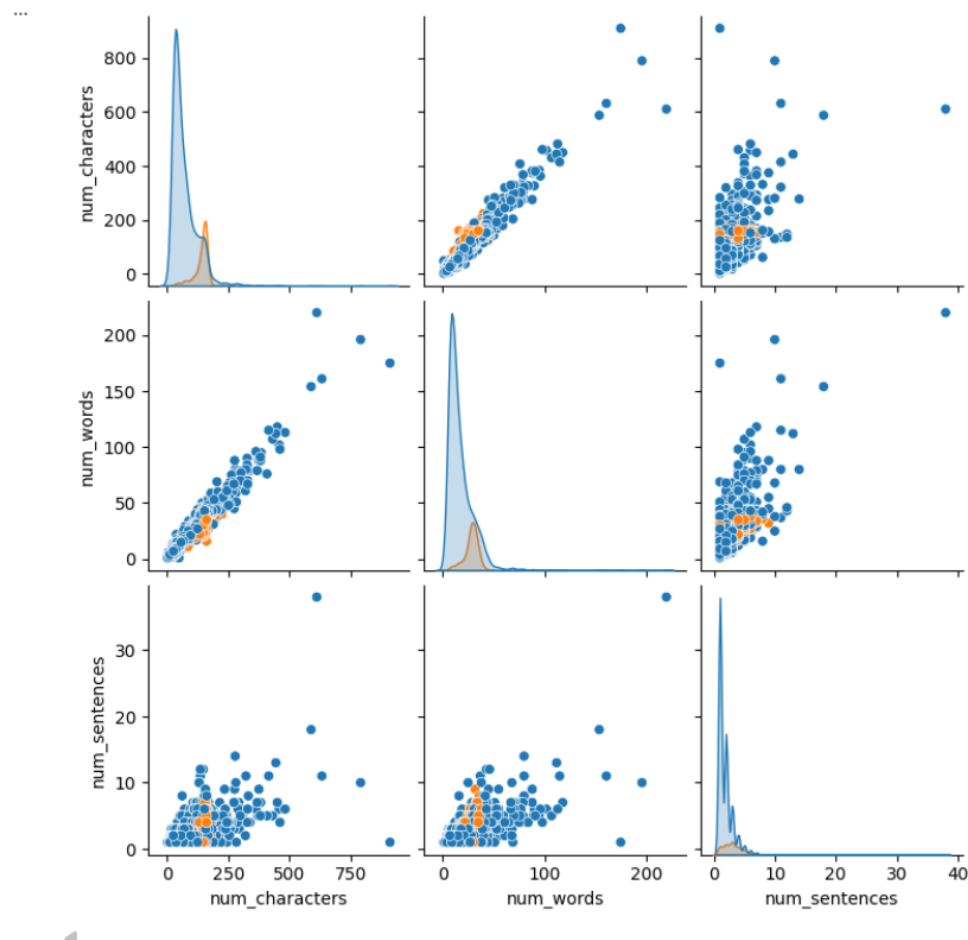
- Histograms were created to compare the number of words in spam and ham messages.
- Spam messages tended to contain more words on average.



The histogram compared the number of words in Ham and Spam messages. It showed that ham messages generally contained fewer words, while spam messages tended to contain a higher number of words. Most spam messages were concentrated between 20 and 35 words, whereas ham messages mostly contained 5 to 20 words. This indicated that the number of words was an important feature for distinguishing spam from ham messages.

6. Pair Plot Analysis

- Pair plot generated using the numerical features.



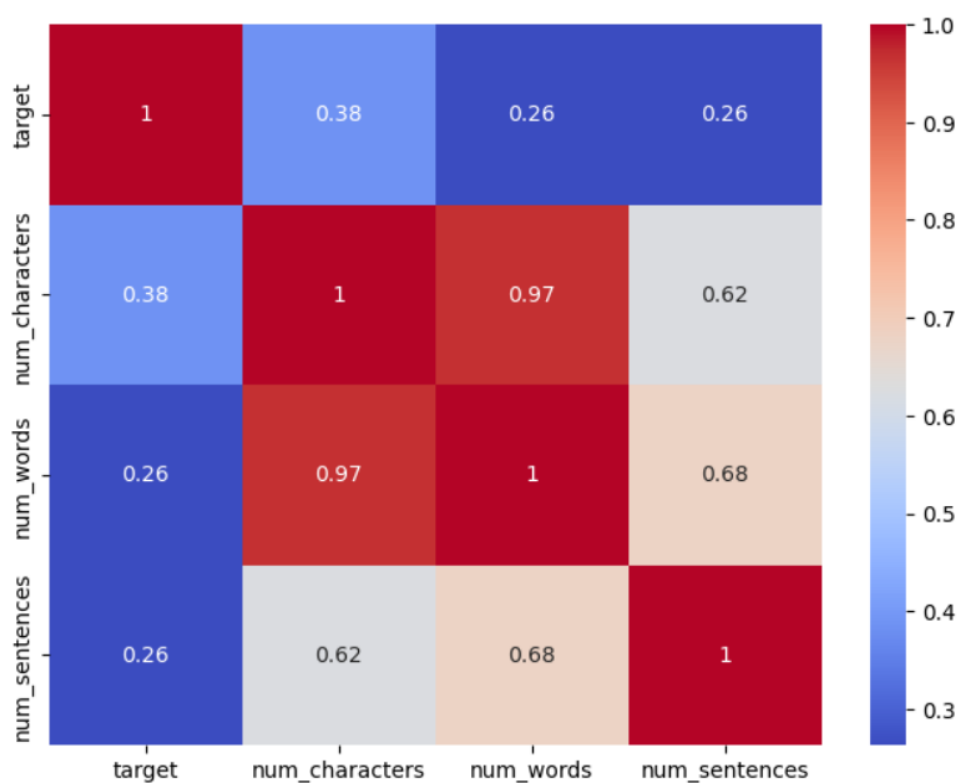
The pair plot shows the relationships among the numerical features: number of characters, number of words, and number of sentences. It indicates a positive relationship between the number of characters and the number of words, as messages with more characters generally contain more words. The plot also shows that spam messages tend to have higher values for these features than ham messages. These patterns suggest that the numerical features are useful for distinguishing spam from ham messages.

7. Correlation Heatmap Analysis

The correlation heatmap **shows** the relationships between the numerical features in the dataset. It **indicates** a strong positive correlation (**0.97**) between the number of characters and the number of words, meaning that longer messages generally contain more words. The number of sentences also **shows** a moderate positive correlation with both characters (**0.62**) and words (**0.68**). The target variable **has** a weak positive correlation with these features, suggesting that message length contributes to spam detection but is not the only determining factor.

```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```

✓ 0.4s



3. Text Preprocessing

The SMS messages were preprocessed using Natural Language Processing (NLP) techniques to prepare the text for machine learning. The following preprocessing steps were applied:

- All text was converted to lowercase to maintain consistency.
- The messages were tokenized into individual words.
- Special characters and non-alphanumeric values were removed.
- English stop words and punctuation were removed to eliminate unnecessary words.
- Porter Stemming was applied to reduce words to their root form (e.g., *playing* → *play*, *loved* → *love*).

```
df.head()
```

[82] ✓ 0.0s

...

	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazy avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkly comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c already say
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe usf live around though

▷ # make the most repitative words bold

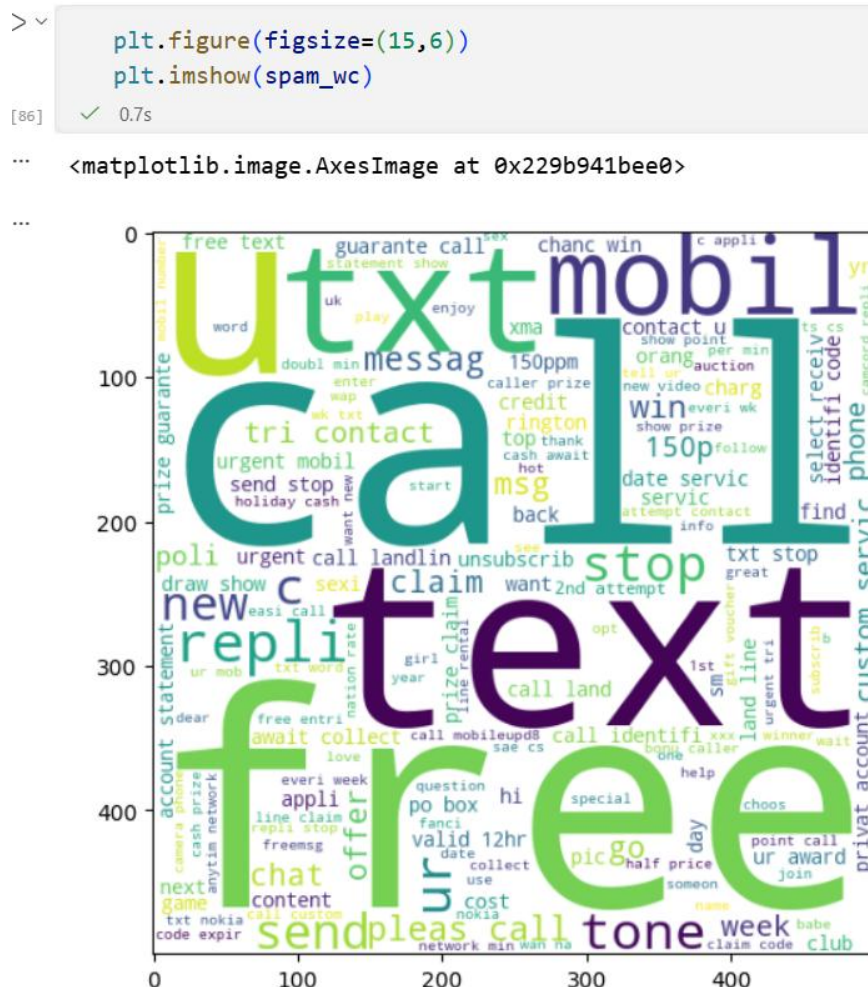
These preprocessing steps reduced noise, standardized the text, and improved the quality of features used for spam classification.

4. Feature Extraction

The cleaned text was converted into numerical form using the TF-IDF Vectorizer.

a. Spam Message Word Cloud

The word cloud shows the most frequently occurring words in spam messages. Words such as "call," "free," "txt," "text," "mobil," "claim," "reply," and "stop" appear more prominently, indicating their higher frequency. These words reflect the promotional and advertising nature of spam messages and help identify common spam-related patterns.



b. Ham Message Word Cloud

The word cloud shows the most frequently occurring words in ham (non-spam) messages. Common words such as "u," "go," "come," "love," "ok," "know," "time," and "good" appear more prominently. These words represent everyday conversations and personal communication, highlighting the difference between legitimate and spam messages.

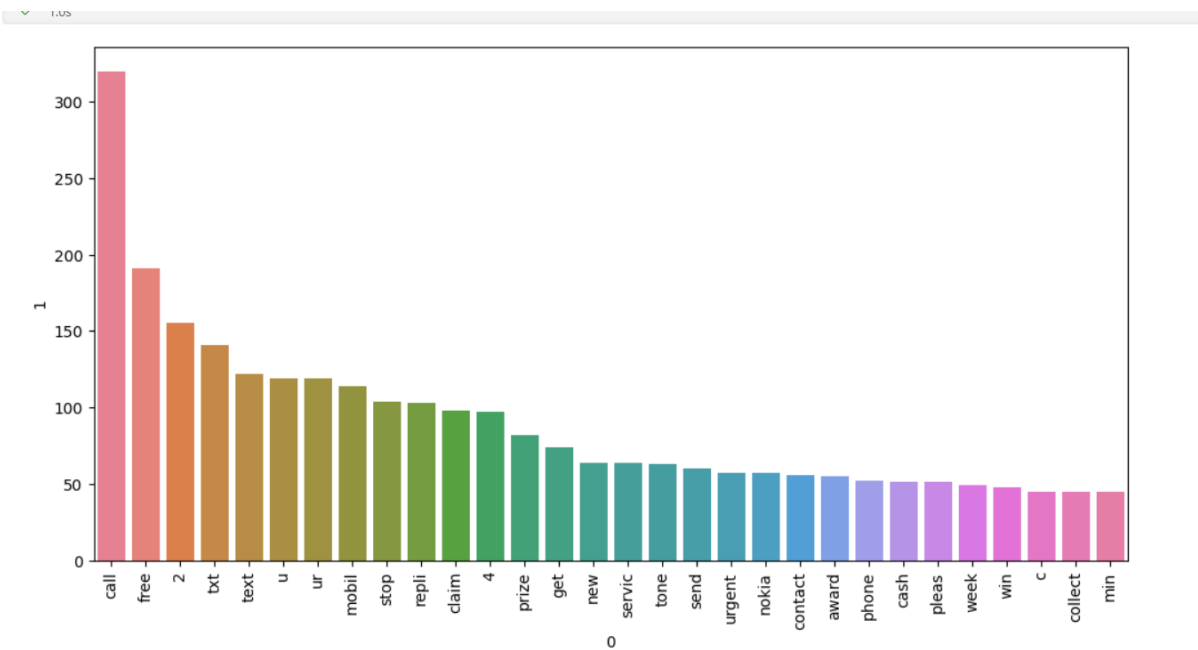
```
plt.figure(figsize=(15,6))
plt.imshow(ham_wc)
```

✓ 0.5s

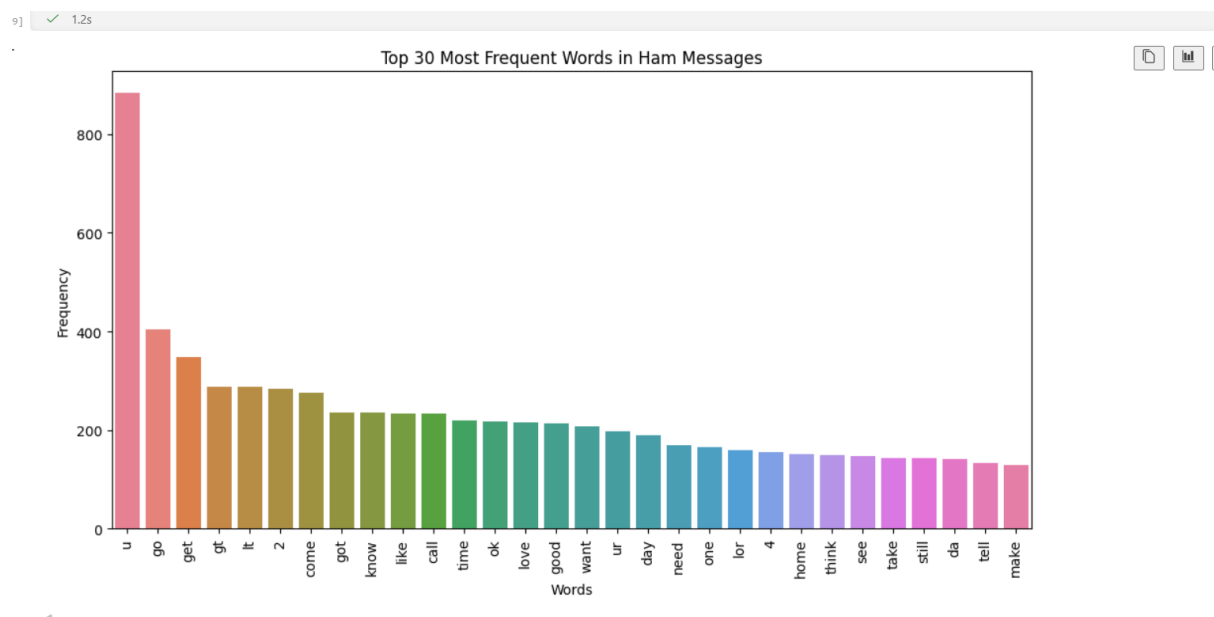
```
<matplotlib.image.AxesImage at 0x229b8b51720>
```



Visualization: Top 30 Most Frequent Words in Spam Messages



Visualization: Top 30 Most Frequent Words in Ham Messages



4. Model Building

The preprocessed SMS messages were converted into numerical features using the **TF-IDF Vectorizer**. The dataset was then divided into **80% training data** and **20% testing data** to train and evaluate the models.

Three Naïve Bayes classifiers were implemented and compared:

- **Gaussian Naïve Bayes (GNB)**
- **Multinomial Naïve Bayes (MNB)**
- **Bernoulli Naïve Bayes (BNB)**

The performance of each model was evaluated using Accuracy, Precision, and the Confusion Matrix. Among the three models, Multinomial Naïve Bayes achieved the best performance and was selected as the baseline model for the spam detection system.

5. Model Evaluation

The three Naïve Bayes classifiers were evaluated using **Accuracy**, **Precision**, and the **Confusion Matrix**. Since the SMS spam dataset is **imbalanced**, **precision** was considered a more important evaluation metric than accuracy. A high precision score indicates that the model correctly identifies spam messages while minimizing false positive predictions (i.e., legitimate messages incorrectly classified as spam).

The evaluation results are summarized below:

Model	Accuracy	Precision
Gaussian Naïve Bayes	87.33%	51.60%
Multinomial Naïve Bayes	97.10%	100.00%
Bernoulli Naïve Bayes	98.36%	99.19%

6. Model Selection

Although **Bernoulli Naïve Bayes** achieved the highest accuracy (**98.36%**), **Multinomial Naïve Bayes** was selected as the final baseline model because it achieved a **precision of 100%**. In an imbalanced spam detection problem, precision is more critical than accuracy because it reduces the number of legitimate (ham) messages that are incorrectly classified as spam. Therefore, **Multinomial Naïve Bayes** was chosen as the baseline model for further development.

7. Advanced Model Development

To further improve the performance of the SMS Spam Detection system, two advanced machine learning algorithms were implemented and evaluated using the TF-IDF feature vectors.

The models used were:

- **Support Vector Machine (SVM)**
- **Random Forest (RF)**

Both models were evaluated using **Accuracy** and **Precision**. The results are presented below.

Model	Accuracy	Precision
Support Vector Machine (SVM)	97.58%	97.48%
Random Forest (RF)	97.68%	97.50%

Discussion

Although Random Forest achieved a slightly higher accuracy (97.68%), Multinomial Naïve Bayes remained the preferred model because it achieved 100% precision, meaning it did not incorrectly classify any legitimate (ham) messages as spam. Since the SMS spam dataset is imbalanced, precision is more important than a small improvement in accuracy. Therefore, Multinomial Naïve Bayes performed consistently well across all evaluation criteria and remained the final model selected for deployment.

8. Model Saving

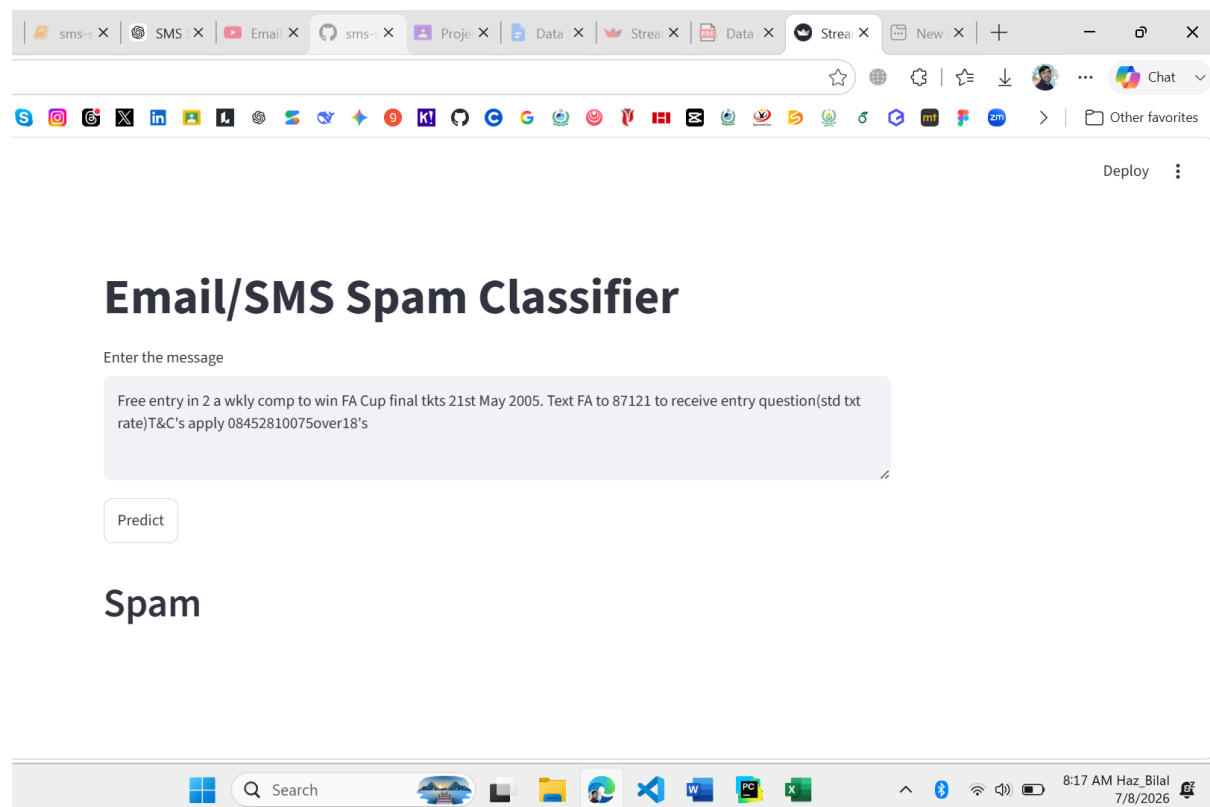
The final **Multinomial Naïve Bayes** model and the **TF-IDF Vectorizer** were saved using the **Pickle** library for deployment.

Saved Files:

- model.pkl
- vectorizer.pkl

9. Deployment

The final model was deployed as an interactive **Streamlit web application**. The application allows users to **enter an SMS or email message** into the text box and click the **"Predict"** button. The input text is first preprocessed and converted into TF-IDF features, after which the trained model predicts whether the message is **Spam** or **Not Spam (Ham)**. This provides users with a simple and real-time spam detection system.



Conclusion

This project successfully developed an SMS Spam Detection System using Natural Language Processing (NLP) and Machine Learning techniques. The dataset was cleaned, explored, and preprocessed before converting the text into numerical features using the TF-IDF Vectorizer.

Three Naïve Bayes classifiers were evaluated, and Multinomial Naïve Bayes was selected as the final model because it achieved 100% precision, making it the most suitable choice for the imbalanced dataset.

Advanced models, including Support Vector Machine (SVM) and Random Forest (RF), were also implemented for comparison, but Multinomial Naïve Bayes remained the preferred model due to its superior precision.

Finally, the trained model was deployed as a Streamlit web application, allowing users to classify SMS or email messages as Spam or Not Spam (Ham) in real time. The project demonstrated the complete end-to-end data science pipeline, from data preprocessing to model deployment, and provided an effective solution for spam message detection.

THE END